Proves

R4 Cheng

November 17, 2025

Cost function

L_2:

$$J(\vec{w}) = \frac{1}{2} \sum_{(\vec{x},y)\in D} (f_{\vec{w}}(\vec{x}) - y)^2 + \lambda \|\vec{w}\|_2^2$$

> Euclidean norm: $\|\vec{w}\|_2 = \sqrt{w_0^2 + w_1^2 + \cdots + w_n^2}$

L_1:

$$J(\vec{w}) = \frac{1}{2} \sum_{(\vec{x},y)\in D} (f_{\vec{w}}(\vec{x}) - y)^2 + \lambda \|\vec{w}\|_1^2$$

> Manhattan norm: $\|\vec{w}\|_1 = |w_0| + |w_1| + \cdots + |w_n|$

Target:

$$\min_{\vec{w}} J(\vec{w})$$

(Batch) Gradient Descent

Start from an arbitrary initial weight vector $\vec{w}$.

$$\vec{w'} \leftarrow \vec{w} - \alpha \Delta J(\vec{w}) = \vec{w} - \alpha \frac{\partial}{\partial \vec{w}} J(\vec{w})$$

> α is called learning rate

And for a single training example $(\vec{x}, y)$, we have:

$$\vec{w'} \leftarrow \vec{w} - \alpha(\vec{w} \cdot \vec{x} - y)\vec{x} = \vec{w} + \alpha(y - \vec{w} \cdot \vec{x})\vec{x}$$

Aka. the least mean square (LMS) udpate rule.

> vs perceptron: $\vec{w'} = \vec{w} + y \cdot \vec{x}$

$$
\begin{aligned}
Derivation: \frac{\partial}{\partial \vec{w}} J(\vec{w}) &= \frac{\partial}{\partial \vec{w}} \frac{1}{2} (f(\vec{x}) - y)^2 \\
&= 2 \cdot \frac{1}{2} (f(\vec{x}) - y) \cdot \frac{\partial}{\partial \vec{w}} (f(\vec{x}) - y) \quad \text{// Chain Rule} \\
&= (\vec{w} \cdot \vec{x} - y) \cdot \frac{\partial}{\partial \vec{w}} (\vec{w} \cdot \vec{x} - y) \\
&= (\vec{w} \cdot \vec{x} - y) \vec{x}
\end{aligned}
$$

Therefore, for the whole dataset D:

$$\vec{w'} \leftarrow \vec{w} + \alpha \sum_{(\vec{x}, y) \in D} (y - \vec{w} \cdot \vec{x}) \vec{x}$$

Stochastic Gradient Descent

Streamline BGD:

$$\vec{w'} \leftarrow \vec{w} + \alpha (y_i - \vec{w} \cdot \vec{x}_i) \vec{x}_i$$

Where $(\vec{x}_i, y_i)$ is a randomly picked training example from D.

Mini-Batch Gradient Descent

> Balance between GD and SGD

$$\vec{w'} \leftarrow \vec{w} + \alpha \sum_{(\vec{x}, y) \in D_m} (y - \vec{w} \cdot \vec{x}) \vec{x}$$

Where $D_m \subset D$ is a randomly picked m examples of training data.

Ridge regression gradient descent

> L_2, becuase it is differentiable

$$\vec{w'} \leftarrow \vec{w} + \alpha \sum_{(\vec{x}, y) \in D} (y - \vec{w} \cdot \vec{x}) \vec{x} - \lambda \vec{w}$$

The function of $\lambda \vec{w}$ is to decrease the magnitude of $\vec{w}$, leading to reduced model complexity and overfitting.

Normal Equation

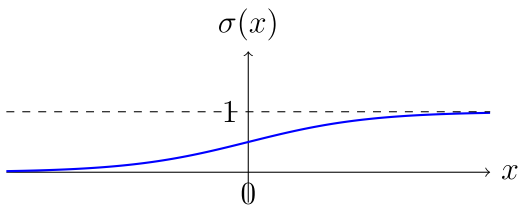
TODO

Deep Learning

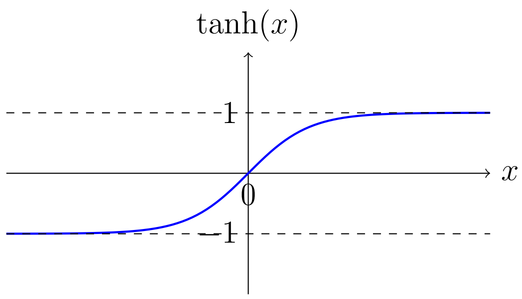
Activation Functions

The activation function introduces non-linearity into the model (network), enabling it to learn and represent complex patterns in the data.

- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}} \Rightarrow$ Usually for binary classification questions.



- Hyperbolic Tangent (Tanh): $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \Rightarrow$ Make data zero-centered which can speed up convergence.



- Rectified Linear Unit (ReLU): $\mathrm{ReLU}(x) = \max(0, x) \Rightarrow$ Only allows positive values to pass through.